

JavaScript

Complete Notes — From Basics to Advanced

Variables · Functions · DOM · Events · ES6+ · OOP · Projects · Job Ready

Prepared by	Ranjiit Sir
Organization	ForkInfosystems, Karad
Contact	9172135355 / 7757962804
Pre-requisite	HTML & CSS (Completed)
Target	Graduation / PG — Job Preparation
Modules	12 Modules + Homework PDF
Year	2026

Table of Contents

Module 1 — Introduction to JavaScript

■ What is JS · How browsers run JS · script tag · console · alert/prompt

Module 2 — Variables, Data Types & Operators

■ var/let/const · 7 data types · typeof · arithmetic/comparison/logical operators

Module 3 — Control Flow — if / else / switch / loops

■ if-else · switch · ternary · for · while · do-while · break · continue

Module 4 — Functions

■ Declaration · Expression · Arrow · Parameters · Return · Scope · Closures

Module 5 — Arrays

■ Create · Access · 20+ methods · spread · destructuring · forEach/map/filter/reduce

Module 6 — Objects

■ Create · Properties · Methods · this · Destructuring · Spread · Object methods

Module 7 — DOM Manipulation

■ Select elements · Change content/style · Create/remove nodes · classList

Module 8 — Events

■ addEventListener · Mouse/Keyboard/Form events · Event object · Bubbling

Module 9 — ES6+ Modern JavaScript

■ Template literals · Destructuring · Modules · Promises · async/await · fetch API

Module 10 — Object-Oriented Programming

■ Constructor functions · Prototypes · Classes · Inheritance · Encapsulation

Module 11 — Error Handling & Local Storage

■ try/catch/finally · throw · localStorage · sessionStorage · JSON

Module 12 — Mini & Major Projects

■ To-Do App · Quiz App · Calculator · Weather App using API

Homework — Module-wise Homework & Interview Questions

■ Theory + Practical per module · Top 35 JS Interview Q&A;

Module 1 — Introduction to JavaScript

1.1 What is JavaScript?

JavaScript (JS) is a lightweight, interpreted **programming language** that makes web pages **interactive and dynamic**. It is the only programming language that runs natively in all web browsers. Together with HTML and CSS, JavaScript forms the three core technologies of the web.

Technology	Role	Example
HTML	Structure	Headings, paragraphs, forms, images
CSS	Appearance	Colours, fonts, layout, animations
JavaScript	Behaviour	Clicks, form validation, API calls, live updates

1.2 Where JavaScript Runs

Environment	Description
Browser (Client)	Runs inside Chrome, Firefox, Edge etc. — most common.
Node.js (Server)	Runs on the server without a browser. Used for backend.
React Native	Builds mobile apps using JavaScript.

1.3 Adding JavaScript to a Web Page

There are three ways to include JavaScript in HTML, just like CSS.

```
<!-- 1. INLINE — inside an HTML attribute (avoid for large code) -->
<button onclick="alert('Hello!')">Click Me</button>

<!-- 2. INTERNAL — inside <script> tag in the HTML file -->
<script>
  alert("Welcome to ForkInfosystems!");
  console.log("Page loaded successfully.");
</script>

<!-- 3. EXTERNAL — separate .js file (BEST PRACTICE) -->
<!-- Put <script> just before </body> so HTML loads first -->
<script src="script.js"></script>

<!-- In script.js: -->
// console.log("External JS file loaded.");
```

Tip: Always place your `<script>` tag just before the closing `</body>` tag. This ensures HTML is fully loaded before JavaScript runs.

1.4 The Browser Console

The **Console** is a developer tool built into every browser. It lets you run JavaScript instantly and see output/errors. Open it with **F12** → Console tab.

```
// Output to console – your most-used debugging tool
console.log("Hello, World!");
console.log("Name:", "Ranjit", "| Age:", 35);

// Other console methods
console.warn("This is a warning!");
console.error("This is an error!");
console.table([{name:"Aarav", marks:88}, {name:"Priya", marks:92}]);
console.clear(); // clears the console
```

1.5 Output Methods

```
// 1. console.log – output to browser console (dev use)
console.log("ForkInfosystems, Karad");

// 2. alert – popup dialog box (stops execution until OK clicked)
alert("Welcome!");

// 3. prompt – popup that asks user for input (returns a string)
let name = prompt("Enter your name:");
console.log("Hello,", name);

// 4. confirm – popup with OK / Cancel (returns true or false)
let answer = confirm("Are you sure?");
if (answer) {
  console.log("User clicked OK");
}

// 5. document.write – writes directly into HTML page
document.write("<h2>Hello from JavaScript!</h2>");

// 6. innerHTML – change content of an element (most used)
document.getElementById("output").innerHTML = "Hello World!";
```

1.6 Comments in JavaScript

```
// This is a single-line comment – browser ignores it

/*
  This is a multi-line comment.
  Use for explaining blocks of code.
*/

// Best practice: write comments to explain WHY, not just WHAT
let speed = 0;    // speed in km/h – starts at rest
```

Module 2 — Variables, Data Types & Operators

2.1 Variables — var, let, const

Variables are **named containers** that store data. JavaScript has three ways to declare variables.

	Re-assign	Hoisted

```
// var – function scoped (avoid in modern JS)
var city = "Karad";
var city = "Pune";    // allowed – no error

// let – block scoped, value can change
let age = 25;
age = 26;              // OK – re-assign allowed
// let age = 30;      // Error – cannot re-declare

// const – block scoped, value CANNOT change
const PI = 3.14159;
// PI = 3;           // Error – Assignment to constant!

// const with objects/arrays: object itself cannot be replaced,
// but its contents CAN be changed
const student = { name: "Aarav", marks: 88 };
student.marks = 92;    // OK – modifying property is allowed
// student = {};      // Error – cannot reassign const

// Naming rules
let firstName = "Ranjit";    // camelCase – recommended
let _private = "hidden";    // underscore prefix – allowed
let $price = 299;           // dollar prefix – allowed
// let 2name = "bad";        // Error – cannot start with number
// let my-var = "bad";       // Error – hyphens not allowed
```

2.2 Data Types

JavaScript has **8 data types**. The first 7 are **primitive** (stored by value). Objects are **reference types** (stored by reference).

Type	Example	Description
String	'Hello' "World"	Text. Use single, double, or backtick quotes.
Number	42 3.14 -7 Infinity	All numbers — integers and decimals.
BigInt	9007199254740991n	Very large integers (add n at end).
Boolean	true false	Only two values — true or false.
undefined	undefined	Variable declared but not assigned.
null	null	Intentional absence of any value.
Symbol	Symbol('id')	Unique identifier (advanced use).
Object	{name:'Aarav'} [1,2,3]	Collection of key-value pairs or array.

```
// String
let name = "Ranjit Sir";
let city = 'Karad';
let message = `Hello, ${name}!`; // Template literal (ES6) – uses backtick

// Number
let marks = 88;
let price = 299.50;
let neg = -15;
let inf = Infinity;
let nan = NaN; // Not a Number (result of invalid math)

// Boolean
let isPassed = true;
let isAdmin = false;

// undefined
let result;
console.log(result); // undefined – declared but no value

// null
let data = null; // deliberately empty

// typeof – check data type
console.log(typeof name); // "string"
console.log(typeof marks); // "number"
console.log(typeof isPassed); // "boolean"
console.log(typeof result); // "undefined"
console.log(typeof null); // "object" ← known JS quirk!
console.log(typeof {}); // "object"
console.log(typeof []); // "object" (arrays are objects)
```

2.3 Type Conversion

```
// String to Number
let str = "42";
let num1 = Number(str);    // 42   (explicit)
let num2 = parseInt(str);  // 42   (integer only)
let num3 = parseFloat("3.14"); // 3.14
let num4 = +str;           // 42   (unary + shortcut)
let bad = Number("hello"); // NaN  (not a valid number)

// Number to String
let n = 100;
let s1 = String(n);        // "100"
let s2 = n.toString();     // "100"
let s3 = n.toString(2);    // "1100100" (binary)
let s4 = n.toFixed(2);     // "100.00"

// To Boolean
Boolean(0);    // false
Boolean("");   // false
Boolean(null); // false
Boolean(undefined); // false
Boolean(NaN);  // false
// Everything else → true
Boolean(1);    // true
Boolean("hello"); // true
Boolean([]);   // true (empty array is truthy!)
```

2.4 Operators

A) Arithmetic Operators

```
let a = 10, b = 3;

console.log(a + b); // 13 (addition)
console.log(a - b); // 7  (subtraction)
console.log(a * b); // 30 (multiplication)
console.log(a / b); // 3.333... (division)
console.log(a % b); // 1   (modulus – remainder)
console.log(a ** b); // 1000 (exponentiation – 10 to power 3)

// Increment / Decrement
let x = 5;
x++;    // x becomes 6 (post-increment)
++x;    // x becomes 7 (pre-increment)
x--;    // x becomes 6 (post-decrement)

// String concatenation with +
let greet = "Hello " + "World"; // "Hello World"
let mix = "Score: " + 88;       // "Score: 88" (number converts to string)
```

B) Comparison Operators

```
console.log(5 == "5");    // true  - loose equality (converts types)
console.log(5 === "5");   // false - strict equality (type must match)
console.log(5 != "5");    // false - loose inequality
console.log(5 !== "5");   // true  - strict inequality
console.log(10 > 5);      // true
console.log(10 < 5);      // false
console.log(10 >= 10);    // true
console.log(10 <= 9);     // false

// ALWAYS use === and !== in real code
// == does type coercion which leads to unexpected results
console.log(0 == false);  // true  ← confusing!
console.log(0 === false); // false ← correct behaviour
```

C) Logical Operators

```
// && (AND) - true only if BOTH are true
console.log(true && true);  // true
console.log(true && false); // false
console.log(5 > 2 && 10 > 3); // true

// || (OR) - true if AT LEAST ONE is true
console.log(false || true); // true
console.log(false || false); // false

// ! (NOT) - flips the value
console.log(!true);         // false
console.log(!false);        // true

// Nullish Coalescing ?? - returns right side if left is null/undefined
let user = null;
let displayName = user ?? "Guest"; // "Guest"

// Logical Assignment
let score = null;
score ??= 0; // score becomes 0 (was null)
score ||= 10; // score stays 0 (0 is falsy, but ??= already ran)
score &&= score * 2; // if score is truthy, double it
```

D) Assignment Operators

```
let n = 10;
n += 5;    // n = n + 5 → 15
n -= 3;    // n = n - 3 → 12
n *= 2;    // n = n * 2 → 24
n /= 4;    // n = n / 4 → 6
n %= 4;    // n = n % 4 → 2
n **= 3;   // n = n ** 3 → 8
```

E) String Operators & Template Literals

```
let first = "Fork";
let last  = "Infosystems";

// Concatenation (old way)
let org = first + last + ", Karad";

// Template Literals (modern – use backticks)
let name  = "Ranjit";
let phone = "9172135355";
let info  = `Instructor: ${name} | Contact: ${phone}`;
// Can include expressions
let price = 500;
let gst   = `Total with GST: ₹${price * 1.18}`;
// Multi-line strings
let html = `
  <div>
    <h2>${name}</h2>
    <p>${org}</p>
  </div>
`;
```

Module 3 — Control Flow: if / else / switch / Loops

3.1 if / else if / else

```
// Basic if
let marks = 75;
if (marks >= 40) {
  console.log("Passed");
}

// if-else
if (marks >= 40) {
  console.log("Passed");
} else {
  console.log("Failed");
}

// if-else if-else (multi-condition)
if (marks >= 90) {
  console.log("Grade: A+");
} else if (marks >= 75) {
  console.log("Grade: A");
} else if (marks >= 60) {
  console.log("Grade: B");
} else if (marks >= 40) {
  console.log("Grade: C");
} else {
  console.log("Grade: F – Failed");
}

// Nested if
let age = 20;
let hasID = true;
if (age >= 18) {
  if (hasID) {
    console.log("Entry allowed");
  } else {
    console.log("Please bring ID");
  }
} else {
  console.log("Entry not allowed – underage");
}
```

3.2 Ternary Operator

A shortcut for simple if-else — one line.

```
// Syntax: condition ? valueIfTrue : valueIfFalse
let age    = 20;
let status = age >= 18 ? "Adult" : "Minor";
console.log(status);    // "Adult"

// Use in template literals
let score = 85;
console.log(`Result: ${score >= 40 ? "PASS" : "FAIL"}`);

// Chained ternary (keep simple – max 2 levels)
let grade = score >= 90 ? "A+" : score >= 75 ? "A" : "B";
```

3.3 switch Statement

Better than many else-if when testing ONE variable against many values.

```
let day = 3;

switch (day) {
  case 1:
    console.log("Monday");
    break;           // ALWAYS add break – stops falling through
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  case 6:
  case 7:           // Multiple cases – same action
    console.log("Weekend!");
    break;
  default:         // Runs if no case matched
    console.log("Invalid day");
}

// Switch with strings
let course = "HTML";
switch (course) {
  case "HTML":
  case "CSS":
    console.log("Web Design course");
    break;
  case "JavaScript":
    console.log("Programming course");
    break;
  default:
    console.log("Course not found");
}
```

3.4 for Loop

```
// Basic for loop
// for (initialisation; condition; update)
for (let i = 1; i <= 5; i++) {
  console.log("Count:", i);
}
// Output: 1 2 3 4 5

// Counting down
for (let i = 10; i >= 1; i--) {
  console.log(i);
}

// Looping through an array
let fruits = ["Apple", "Mango", "Banana", "Orange"];
for (let i = 0; i < fruits.length; i++) {
  console.log(i, fruits[i]);
}

// for...of – modern loop for arrays (ES6)
for (let fruit of fruits) {
  console.log(fruit);
}

// for...in – loop over object keys
let student = { name: "Aarav", age: 20, marks: 88 };
for (let key in student) {
  console.log(key, ":", student[key]);
}
// Output: name : Aarav    age : 20    marks : 88
```

3.5 while and do...while Loops

```
// while – checks condition BEFORE running
let count = 1;
while (count <= 5) {
  console.log("while count:", count);
  count++;      // IMPORTANT: update or infinite loop!
}

// do...while – runs at LEAST ONCE, checks condition AFTER
let num = 1;
do {
  console.log("do-while num:", num);
  num++;
} while (num <= 5);

// Practical: keep asking until valid input
let userInput;
do {
  userInput = prompt("Enter a number between 1-10:");
} while (userInput < 1 || userInput > 10);
console.log("Valid input:", userInput);
```

3.6 break and continue

```
// break – exit the loop immediately
for (let i = 1; i <= 10; i++) {
  if (i === 5) break;    // Stop when i equals 5
  console.log(i);        // Prints: 1 2 3 4
}

// continue – skip current iteration, go to next
for (let i = 1; i <= 10; i++) {
  if (i % 2 === 0) continue; // Skip even numbers
  console.log(i);            // Prints: 1 3 5 7 9
}

// Labelled break (nested loops)
outer: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (j === 1) break outer; // Breaks both loops
    console.log(i, j);
  }
}
```

Module 4 — Functions

4.1 What is a Function?

A function is a **reusable block of code** that performs a specific task. Functions help avoid repetition (DRY — Don't Repeat Yourself). JavaScript has several ways to define a function.

4.2 Function Declaration

```
// Syntax
function functionName(parameters) {
  // code to run
  return value; // optional
}

// Example
function greet(name) {
  return "Hello, " + name + "!";
}

// Calling (invoking) the function
let message = greet("Ranjit");
console.log(message); // Hello, Ranjit!

// Function with multiple parameters
function add(a, b) {
  return a + b;
}
console.log(add(10, 25)); // 35

// Function without return (returns undefined)
function showInfo(name, phone) {
  console.log(`Name: ${name} | Phone: ${phone}`);
}
showInfo("Ranjit Sir", "9172135355");

// Function declarations are HOISTED — can call before declaring!
sayHello(); // Works!
function sayHello() {
  console.log("Hello from ForkInfosystems!");
}
```

4.3 Function Expression

```
// Store a function in a variable
const greet = function(name) {
  return `Hello, ${name}!`;
};
console.log(greet("Aarav"));

// Function expressions are NOT hoisted
// greet2(); // Error – cannot call before declaration
const greet2 = function() { return "Hi!"; };

// Named function expression (useful for recursion/debugging)
const factorial = function fact(n) {
  if (n <= 1) return 1;
  return n * fact(n - 1); // can use 'fact' inside itself
};
console.log(factorial(5)); // 120
```

4.4 Arrow Functions (ES6)

Arrow functions are a shorter syntax for function expressions. They are the most modern and widely used style today.

```
// Traditional function
function add(a, b) { return a + b; }

// Arrow function – step by step shorter forms:
const add1 = (a, b) => { return a + b; }; // full arrow
const add2 = (a, b) => a + b;           // implicit return (no braces)
const square = n => n * n;              // single param – no brackets needed
const greet = () => "Hello!";          // no params – empty brackets

// Examples
const double = n => n * 2;
console.log(double(5)); // 10

const isEven = n => n % 2 === 0;
console.log(isEven(4)); // true

// Arrow functions in array methods
let numbers = [1, 2, 3, 4, 5];
let doubled = numbers.map(n => n * 2);
console.log(doubled); // [2, 4, 6, 8, 10]

// Arrow functions and 'this'
// Arrow functions do NOT have their own 'this'
// They inherit 'this' from the surrounding scope
const obj = {
  name: "ForkInfosystems",
  regular: function() { console.log(this.name); }, // works
  arrow: () => { console.log(this.name); },        // undefined!
};
```

4.5 Default Parameters

```
// Without default – missing args become undefined
function greet(name, greeting) {
  return `${greeting}, ${name}!`;
}
console.log(greet("Aarav"));    // "undefined, Aarav!"

// With default parameter values
function greet(name, greeting = "Hello") {
  return `${greeting}, ${name}!`;
}
console.log(greet("Aarav"));    // "Hello, Aarav!"
console.log(greet("Priya", "Namaste")); // "Namaste, Priya!"

// Default can be an expression
function createUser(name, role = "Student", id = Date.now()) {
  return { name, role, id };
}
```

4.6 Rest Parameters & Spread Operator

```
// Rest (...) – collect remaining arguments into an array
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3));    // 6
console.log(sum(10, 20, 30, 40)); // 100

function introduce(first, last, ...hobbies) {
  console.log(`${first} ${last} likes: ${hobbies.join(", ")}`);
}
introduce("Aarav", "Sharma", "Coding", "Gaming", "Cricket");
// Aarav Sharma likes: Coding, Gaming, Cricket

// Spread (...) – expand array/object into individual items
let arr1 = [1, 2, 3];
let arr2 = [4, 5, 6];
let combined = [...arr1, ...arr2];    // [1, 2, 3, 4, 5, 6]
let copy      = [...arr1];            // create a copy

console.log(Math.max(...arr1));    // 3 – spread as arguments
```

4.7 Scope

```
// Global scope – accessible everywhere
let globalVar = "I am global";

function test() {
  // Function/local scope – only inside this function
  let localVar = "I am local";
  console.log(globalVar); // OK – can access global
  console.log(localVar); // OK – in same scope
}
test();
// console.log(localVar); // Error – not accessible outside

// Block scope – let and const are block-scoped
{
  let blockVar = "inside block";
  const blockConst = 42;
  var varVar = "var ignores block scope!";
}
// console.log(blockVar); // Error – blockVar not accessible
console.log(varVar); // OK – var leaks out of block!

// Scope chain – inner can access outer, not vice versa
let outer = "outer value";
function outerFn() {
  let inner = "inner value";
  function innerFn() {
    console.log(outer); // OK – access outer scope
    console.log(inner); // OK – access parent scope
  }
  innerFn();
  // console.log(innerFn local vars); // Not accessible
}
```

4.8 Closures

A closure is a function that **remembers its outer scope** even after the outer function has finished executing. Very important concept for interviews!

```
function makeCounter() {
  let count = 0;           // This variable is 'closed over'
  return function() {
    count++;
    return count;
  };
}

let counter = makeCounter();
console.log(counter());    // 1
console.log(counter());    // 2
console.log(counter());    // 3

// Each call to makeCounter creates a NEW closure
let counter2 = makeCounter();
console.log(counter2());   // 1 – independent of counter1

// Practical closure – private variable
function bankAccount(initialBalance) {
  let balance = initialBalance; // private – cannot access directly
  return {
    deposit: (amt) => { balance += amt; return balance; },
    withdraw: (amt) => { balance -= amt; return balance; },
    getBalance: () => balance,
  };
}

let acc = bankAccount(1000);
console.log(acc.deposit(500)); // 1500
console.log(acc.withdraw(200)); // 1300
// console.log(balance);      // Error – balance is private!
```

Module 5 — Arrays

5.1 Creating & Accessing Arrays

An array is an **ordered collection** of values stored in a single variable. Arrays can hold any data type — including other arrays and objects.

```
// Creating arrays
let fruits   = ["Apple", "Mango", "Banana"];    // array literal
let numbers  = [10, 20, 30, 40, 50];
let mixed    = [1, "hello", true, null, {a:1}]; // mixed types
let empty    = [];                             // empty array
let matrix   = [[1,2],[3,4],[5,6]];            // 2D array

// Accessing elements (index starts at 0)
console.log(fruits[0]);    // "Apple"
console.log(fruits[2]);    // "Banana"
console.log(fruits[fruits.length - 1]); // Last element: "Banana"

// Modifying elements
fruits[1] = "Grapes";
console.log(fruits);      // ["Apple", "Grapes", "Banana"]

// Array length
console.log(fruits.length); // 3

// Destructuring – extract values into variables
let [first, second, ...rest] = fruits;
console.log(first);    // "Apple"
console.log(rest);     // ["Banana"]
```

5.2 Array Methods — Complete Reference

Method	Returns	Description
<code>push(val)</code>	new length	Add to END. Modifies original.
<code>pop()</code>	removed val	Remove from END. Modifies original.
<code>unshift(val)</code>	new length	Add to BEGINNING. Modifies original.
<code>shift()</code>	removed val	Remove from BEGINNING. Modifies original.
<code>splice(i,n,..v)</code>	removed []	Remove n items at index i, optionally insert v.
<code>slice(start,end)</code>	new []	Copy portion. Does NOT modify original.
<code>concat(...arr)</code>	new []	Join arrays together.
<code>join(sep)</code>	string	Join all elements into a string.
<code>indexOf(val)</code>	index/-1	First index of value, or -1 if not found.
<code>lastIndexOf(val)</code>	index/-1	Last index of value.
<code>includes(val)</code>	boolean	Check if array contains value.
<code>find(fn)</code>	value/undef	First element where fn returns true.
<code>findIndex(fn)</code>	index/-1	Index of first element where fn returns true.
<code>filter(fn)</code>	new []	New array of all elements where fn is true.
<code>map(fn)</code>	new []	New array with fn applied to each element.
<code>reduce(fn, init)</code>	single val	Reduce array to one value using fn.
<code>forEach(fn)</code>	undefined	Run fn on each element. No return value.
<code>some(fn)</code>	boolean	True if ANY element satisfies fn.
<code>every(fn)</code>	boolean	True if ALL elements satisfy fn.
<code>sort(fn)</code>	sorted []	Sort in place. Pass fn for custom sort.
<code>reverse()</code>	reversed []	Reverse in place.
<code>flat(depth)</code>	new []	Flatten nested arrays.
<code>flatMap(fn)</code>	new []	map() + flat(1) combined.
<code>fill(val,s,e)</code>	modified []	Fill with value from index s to e.
<code>Array.from()</code>	new []	Create array from iterable (string, NodeList).

Array.isArray()	boolean	Check if value is an array.
------------------------	----------------	-----------------------------

5.3 Array Methods — Code Examples

```
let nums = [3, 1, 4, 1, 5, 9, 2, 6];

// push / pop / shift / unshift
nums.push(7);      // [3,1,4,1,5,9,2,6,7]
nums.pop();        // removes 7
nums.unshift(0);   // [0,3,1,4,...]
nums.shift();      // removes 0

// splice – remove 2 items starting at index 2
let removed = nums.splice(2, 2);    // removes [4,1]
// Insert – at index 2, remove 0, add 10,20
nums.splice(2, 0, 10, 20);

// slice – copy without modifying
let copy = nums.slice(1, 4);        // index 1 to 3

// sort – default sorts as STRINGS!
let unsorted = [10, 9, 100, 21, 3];
unsorted.sort();                    // [10, 100, 21, 3, 9] WRONG!
unsorted.sort((a, b) => a - b);      // [3, 9, 10, 21, 100] Correct
unsorted.sort((a, b) => b - a);      // Descending order

// join
let words = ["Fork", "Info", "systems"];
console.log(words.join(""));        // "ForkInfosystems"
console.log(words.join("-"));       // "Fork-Info-systems"

// includes / indexOf
let arr = [1, 2, 3, 4, 5];
console.log(arr.includes(3));        // true
console.log(arr.indexOf(3));         // 2
console.log(arr.indexOf(10));        // -1
```

5.4 Higher-Order Array Methods (Most Important)

```
let students = [
  { name: "Aarav", marks: 88 },
  { name: "Priya", marks: 92 },
  { name: "Rohan", marks: 56 },
  { name: "Sneha", marks: 78 },
  { name: "Vikram", marks: 35 },
];

// forEach – run function on each element (no return)
students.forEach((s, index) => {
  console.log(`${index + 1}. ${s.name}: ${s.marks}`);
});

// map – transform each element, returns NEW array
let names = students.map(s => s.name);
console.log(names); // ["Aarav", "Priya", "Rohan", "Sneha", "Vikram"]

let grades = students.map(s => ({
  name: s.name,
  grade: s.marks >= 75 ? "A" : s.marks >= 60 ? "B" : "C",
}));

// filter – keep only elements where fn returns true
let passed = students.filter(s => s.marks >= 40);
let toppers = students.filter(s => s.marks >= 80);

// find – first match
let topper = students.find(s => s.marks > 90);
console.log(topper); // { name: "Priya", marks: 92 }

// reduce – collapse to single value
let total = students.reduce((sum, s) => sum + s.marks, 0);
let avg = total / students.length;
console.log(`Average: ${avg}`); // Average: 69.8

// some / every
let anyFailed = students.some(s => s.marks < 40); // true
let allPassed = students.every(s => s.marks >= 40); // false

// Chaining methods
let result = students
  .filter(s => s.marks >= 60) // keep passed students
  .map(s => s.name) // get their names
  .sort(); // sort alphabetically
console.log(result); // ["Aarav", "Priya", "Sneha"]
```

Module 6 — Objects

6.1 Creating Objects

An object is a collection of **key-value pairs** (properties). Objects represent real-world entities and their characteristics.

```
// Object literal (most common)
let student = {
  name: "Aarav Sharma",
  age: 20,
  marks: 88,
  isPassed: true,
  address: { // nested object
    city: "Karad",
    state: "Maharashtra",
  },
  hobbies: ["Coding", "Cricket"], // array inside object

  // Method — a function inside an object
  greet: function() {
    return `Hello, I am ${this.name}`;
  },

  // ES6 shorthand method
  study() {
    return `${this.name} is studying.`;
  },
};

// Accessing properties
console.log(student.name); // dot notation
console.log(student["age"]); // bracket notation (useful for dynamic keys)
console.log(student.address.city); // nested
console.log(student.hobbies[0]); // array inside object

// Calling methods
console.log(student.greet());

// Add / modify / delete properties
student.email = "aarav@email.com"; // add new property
student.marks = 92; // modify
delete student.isPassed; // delete
console.log(student);
```

6.2 this Keyword

```
const person = {
  name: "Ranjit Sir",
  org: "ForkInfosystems",

  introduce() {
    // 'this' refers to the object that called the method
    return `I am ${this.name} from ${this.org}`;
  },

  showDetails: function() {
    console.log(this); // logs the entire 'person' object
  },
};

console.log(person.introduce());

// 'this' in regular functions depends on how they are called
function show() {
  console.log(this); // In browser: Window object. In strict mode: undefined
}

// 'this' in arrow function – inherits from enclosing scope
const obj = {
  name: "Test",
  arrowMethod: () => {
    console.log(this.name); // undefined – arrow has no own 'this'
  },
};
```

6.3 Object Destructuring

```
const student = { name:"Aarav", age:20, marks:88, city:"Karad" };

// Destructuring – extract into variables
const { name, marks } = student;
console.log(name, marks);    // Aarav 88

// Rename while destructuring
const { name: studentName, marks: score } = student;
console.log(studentName, score);    // Aarav 88

// Default values
const { age, grade = "A" } = student;
console.log(grade);    // "A" (student has no 'grade', uses default)

// Nested destructuring
const user = { profile: { firstName: "Priya", city: "Pune" } };
const { profile: { firstName, city } } = user;
console.log(firstName, city);    // Priya Pune

// In function parameters
function display({ name, marks }) {
  console.log(`${name} scored ${marks}`);
}
display(student);    // Aarav scored 88
```

6.4 Spread & Object.assign

```
const base = { a: 1, b: 2 };
const extra = { c: 3, d: 4 };

// Merge objects with spread
const merged = { ...base, ...extra };    // { a:1, b:2, c:3, d:4 }

// Copy an object
const copy = { ...base };

// Override property
const updated = { ...base, b: 99 };    // { a:1, b:99 }

// Object.assign – same as spread (older way)
const merged2 = Object.assign({}, base, extra);

// Practical: update user settings
const defaults = { theme:"light", lang:"en", fontSize:16 };
const userPrefs = { theme:"dark", fontSize:18 };
const settings = { ...defaults, ...userPrefs };
// { theme:"dark", lang:"en", fontSize:18 }
```

6.5 Useful Object Methods

```
const product = { id:1, name:"Laptop", price:45000, inStock:true };

// Object.keys – array of keys
console.log(Object.keys(product));
// ["id","name","price","inStock"]

// Object.values – array of values
console.log(Object.values(product));
// [1,"Laptop",45000,true]

// Object.entries – array of [key, value] pairs
for (let [key, val] of Object.entries(product)) {
  console.log(`${key} = ${val}`);
}

// Object.freeze – prevent modification
const config = Object.freeze({ apiUrl: "https://api.example.com", timeout: 3000 });
config.timeout = 5000; // Silently fails (no error in non-strict mode)
console.log(config.timeout); // still 3000

// Computed property names
let field = "name";
let obj = { [field]: "Dynamic!" }; // { name: "Dynamic!" }

// Shorthand properties
let name = "Aarav";
let marks = 92;
let st = { name, marks }; // { name:"Aarav", marks:92 }
// Same as: { name: name, marks: marks }

// Optional chaining ?.
let user = { address: { city: "Karad" } };
console.log(user?.address?.city); // "Karad"
console.log(user?.phone?.number); // undefined (no error!)
```

Module 7 — DOM Manipulation

7.1 What is the DOM?

The **DOM (Document Object Model)** is a tree-like representation of your HTML page that JavaScript can read and modify. When the browser loads HTML, it creates this tree — JavaScript can then change any element, its content, style, or attributes without reloading the page.

7.2 Selecting Elements

```
// getElementById — returns single element (or null)
let heading = document.getElementById("main-title");

// getElementsByClassName — returns HTMLCollection (live)
let cards = document.getElementsByClassName("card");

// getElementsByTagName — returns HTMLCollection
let paras = document.getElementsByTagName("p");

// querySelector — returns FIRST match (CSS selector syntax)
let btn = document.querySelector(".btn-primary");
let nav = document.querySelector("nav");
let special = document.querySelector("#hero .cta-btn");

// querySelectorAll — returns ALL matches as NodeList
let allCards = document.querySelectorAll(".card");
let allLinks = document.querySelectorAll("nav a");

// Looping over results
allCards.forEach(card => {
  console.log(card.textContent);
});

// document shortcuts
console.log(document.body);
console.log(document.head);
console.log(document.title);
console.log(document.URL);
```

7.3 Changing Content

```
let el = document.getElementById("output");

// innerHTML – sets HTML (can include tags)
el.innerHTML = "<strong>ForkInfosystems</strong>, Karad";

//.textContent – sets plain text (safer, no HTML parsing)
el.textContent = "Hello World – no HTML tags parsed";

// innerText – visible text only (respects CSS display)
el.innerText = "Visible text";

// value – for form input elements
let input = document.getElementById("username");
input.value = "Aarav";
console.log(input.value); // read the current value
```

7.4 Changing Styles & Classes

```
let box = document.getElementById("box");

// Inline styles – use camelCase for CSS properties
box.style.backgroundColor = "#1a237e";
box.style.color           = "white";
box.style.fontSize        = "20px";
box.style.padding         = "15px 30px";
box.style.borderRadius    = "8px";
box.style.display         = "none";    // hide
box.style.display         = "block";   // show

// className – set the class attribute
box.className = "card active"; // replaces ALL classes

// classList – manage classes individually (PREFERRED)
box.classList.add("active");    // add class
box.classList.remove("hidden"); // remove class
box.classList.toggle("dark-mode"); // add if absent, remove if present
box.classList.contains("active"); // returns true/false
box.classList.replace("old", "new"); // replace one class with another

// Reading computed styles
let style = window.getComputedStyle(box);
console.log(style.backgroundColor);
```

7.5 Changing Attributes

```
let img = document.querySelector("img");
let link = document.querySelector("a");

// Get attribute
console.log(img.getAttribute("src"));
console.log(img.getAttribute("alt"));

// Set attribute
img.setAttribute("src", "new-image.jpg");
img.setAttribute("alt", "New description");
link.setAttribute("href", "https://forksystems.com");

// Remove attribute
img.removeAttribute("width");

// Check if attribute exists
console.log(img.hasAttribute("alt")); // true

// Direct property access (shorthand for some attrs)
img.src = "another.jpg";
link.href = "https://example.com";
input.disabled = true; // disable a form field
input.disabled = false; // enable it
input.placeholder = "Type here...";
```

7.6 Creating & Removing Elements

```
// Create a new element
let newCard = document.createElement("div");
newCard.className = "card";
newCard.innerHTML = `
  <h3>Aarav Sharma</h3>
  <p>Web Developer</p>
  <a href="#">View Profile</a>
`;

// Add to DOM
let container = document.getElementById("cards-container");
container.appendChild(newCard);           // add at END
container.prepend(newCard);               // add at BEGINNING
container.insertBefore(newCard, container.firstChild);

// insertAdjacentHTML – powerful alternative
container.insertAdjacentHTML("beforeend",
  "<p class='note'>New content added!</p>");
// positions: beforebegin | afterbegin | beforeend | afterend

// Remove elements
let oldItem = document.getElementById("old-item");
oldItem.remove();                         // modern way
oldItem.parentNode.removeChild(oldItem);  // older way

// Clone an element
let clone = newCard.cloneNode(true);      // true = deep clone (with children)
container.appendChild(clone);

// Practical: build a list from array
let students = ["Aarav", "Priya", "Rohan", "Sneha"];
let ul = document.createElement("ul");
students.forEach(name => {
  let li = document.createElement("li");
  li.textContent = name;
  ul.appendChild(li);
});
document.body.appendChild(ul);
```

Module 8 — Events

8.1 What is an Event?

An **event** is something that happens in the browser — a click, keypress, form submit, page load etc. JavaScript can **listen** for these events and run code in response.

8.2 addEventListener (Best Practice)

```
// Syntax:
// element.addEventListener("event", callbackFunction);

let btn = document.getElementById("myButton");

// Method 1: Named function
function handleClick() {
  console.log("Button was clicked!");
}
btn.addEventListener("click", handleClick);

// Method 2: Anonymous function
btn.addEventListener("click", function() {
  console.log("Clicked with anonymous function!");
});

// Method 3: Arrow function (most common in modern JS)
btn.addEventListener("click", () => {
  btn.textContent = "Clicked!";
  btn.style.backgroundColor = "#e65100";
});

// Remove listener (must use named function reference)
btn.removeEventListener("click", handleClick);

// Why addEventListener over onclick?
// addEventListener can attach MULTIPLE handlers
// onclick overwrites any previous handler
btn.onclick = function() { console.log("1"); };
btn.onclick = function() { console.log("2"); }; // replaces 1!
btn.addEventListener("click", () => console.log("A")); // OK
btn.addEventListener("click", () => console.log("B")); // OK — both fire
```

8.3 Common Events — Reference

Category	Event	When it fires
Mouse	click	Single left-click on element
Mouse	dblclick	Double-click on element
Mouse	mouseenter	Mouse enters element (no bubble)
Mouse	mouseleave	Mouse leaves element (no bubble)
Mouse	mouseover	Mouse over element or its children
Mouse	mouseout	Mouse leaves element or children
Mouse	mousemove	Mouse moves over element
Mouse	contextmenu	Right-click (context menu)
Keyboard	keydown	Key is pressed down
Keyboard	keyup	Key is released
Keyboard	keypress	Key press (deprecated, use keydown)
Form	submit	Form is submitted
Form	change	Input value changes & loses focus
Form	input	Input value changes (every keystroke)
Form	focus	Element gains focus
Form	blur	Element loses focus
Window	load	Page fully loaded (images etc.)
Window	DOMContentLoaded	HTML parsed, DOM ready (faster)
Window	resize	Window resized
Window	scroll	Page scrolled
Drag	dragstart	Dragging starts
Drag	drop	Element dropped
Touch	touchstart	Touch begins (mobile)

8.4 The Event Object

```
// The callback receives an event object with useful properties
document.addEventListener("click", function(event) {
  console.log(event.type);           // "click"
  console.log(event.target);         // element that was clicked
  console.log(event.clientX);        // X position of click
  console.log(event.clientY);        // Y position of click
  event.preventDefault();            // stop default behaviour
  event.stopPropagation();           // stop event bubbling
});

// Keyboard events
document.addEventListener("keydown", function(e) {
  console.log(e.key);               // "Enter", "a", "ArrowUp" etc.
  console.log(e.code);              // "KeyA", "Enter", "Space" etc.
  console.log(e.ctrlKey);           // true if Ctrl held
  console.log(e.shiftKey);          // true if Shift held
  if (e.key === "Enter") console.log("Enter pressed!");
});

// Form events
let form = document.getElementById("myForm");
form.addEventListener("submit", function(e) {
  e.preventDefault(); // stop page from reloading!
  let name = document.getElementById("name").value;
  if (!name) {
    alert("Name is required!");
    return;
  }
  console.log("Form submitted with name:", name);
});

// Input events – react on every keystroke
let searchBox = document.getElementById("search");
searchBox.addEventListener("input", (e) => {
  console.log("Searching for:", e.target.value);
});
```

8.5 Event Bubbling & Delegation

```
// EVENT BUBBLING – event fires on child, then bubbles up to parent
document.getElementById("child").addEventListener("click", () => {
  console.log("Child clicked"); // fires first
});
document.getElementById("parent").addEventListener("click", () => {
  console.log("Parent clicked"); // fires second (bubbles up)
});

// Stop bubbling
document.getElementById("child").addEventListener("click", (e) => {
  e.stopPropagation(); // parent handler will NOT fire
  console.log("Child only");
});

// EVENT DELEGATION – attach ONE listener to parent,
// handle events from all children (even dynamically added ones)
document.getElementById("todo-list").addEventListener("click", (e) => {
  // e.target is the actual element clicked
  if (e.target.matches(".delete-btn")) {
    e.target.parentElement.remove();
  }
  if (e.target.matches(".complete-btn")) {
    e.target.parentElement.classList.toggle("completed");
  }
});
// Much better than attaching listeners to every list item!
```

8.6 Practical Example — Interactive Card

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Interactive Demo</title>
  <style>
    body { font-family:Arial,sans-serif; padding:30px; background:#f5f7ff; }
    .card {
      background:white; padding:20px; border-radius:10px;
      box-shadow:0 4px 15px rgba(0,0,0,0.1);
      max-width:400px; margin:0 auto; text-align:center;
    }
    .btn {
      padding:10px 24px; background:#1a237e; color:white;
      border:none; border-radius:6px; cursor:pointer;
      font-size:15px; margin:5px; transition:background 0.3s;
    }
    .btn:hover { background:#e65100; }
    #counter { font-size:48px; font-weight:bold; color:#1a237e; }
    #name-display { margin-top:15px; font-size:18px; color:#333; }
  </style>
</head>
<body>
  <div class="card">
    <h2>JS Events Demo</h2>
    <div id="counter">0</div>
    <button class="btn" id="inc">+ Increment</button>
    <button class="btn" id="dec">- Decrement</button>
    <button class="btn" id="reset">Reset</button>
    <hr style="margin:20px 0">
    <input type="text" id="nameInput" placeholder="Type your name...">
    <div id="name-display">Hello, Guest!</div>
  </div>

  <script>
    // Counter
    let count = 0;
    const display = document.getElementById("counter");

    document.getElementById("inc").addEventListener("click", () => {
      count++;
      display.textContent = count;
      display.style.color = count > 0 ? "#2e7d32" : "#b71c1c";
    });
    document.getElementById("dec").addEventListener("click", () => {
      count--;
      display.textContent = count;
      display.style.color = count > 0 ? "#2e7d32" : "#b71c1c";
    });
    document.getElementById("reset").addEventListener("click", () => {
      count = 0;
      display.textContent = count;
      display.style.color = "#1a237e";
    });
  </script>
</body>
</html>
```

```
});

// Live name display
document.getElementById("nameInput").addEventListener("input", (e) => {
  let val = e.target.value.trim();
  document.getElementById("name-display").textContent =
    val ? `Hello, ${val}!` : "Hello, Guest!";
});
</script>
</body>
</html>
```

Module 9 — ES6+ Modern JavaScript

9.1 Template Literals

```
// Old way: string concatenation
let name = "Ranjit";
let phone = "9172135355";
let old = "Name: " + name + " | Phone: " + phone;

// Template literal – backticks, ${expression}
let modern = `Name: ${name} | Phone: ${phone}`;

// Multi-line strings
let html = `
  <div class="card">
    <h2>${name}</h2>
    <p>ForkInfosystems, Karad</p>
  </div>
`;

// Any expression inside ${}
let a = 10, b = 5;
console.log(`Sum: ${a + b}`); // Sum: 15
console.log(`Max: ${Math.max(a, b)}`); // Max: 10
console.log(`${a > b ? "a" : "b"} wins`); // a wins

// Tagged template literal (advanced)
function highlight(strings, ...values) {
  return strings.reduce((result, str, i) =>
    result + str + (values[i] ? `<b>${values[i]}</b>` : ""), "");
}
let result = highlight`Hello ${name}, you scored ${88}%`;
```

9.2 Destructuring (Array & Object)

```
// Array destructuring
let [x, y, z] = [10, 20, 30];
let [first, , third] = [1, 2, 3]; // skip index 1
let [a = 5, b = 10] = [1]; // default values: a=1, b=10
let [p, ...rest] = [1,2,3,4,5]; // rest: [2,3,4,5]

// Swap variables (no temp variable needed!)
let m = 1, n = 2;
[m, n] = [n, m]; // m=2, n=1

// Object destructuring
let { name, age, city = "Unknown" } = { name:"Aarav", age:20 };
let { name: fullName } = { name: "Priya Patil" }; // rename

// Nested destructuring
let { address: { street, pin } } = {
  address: { street: "MG Road", pin: "415110" }
};

// Function parameter destructuring
function show({ name, marks, grade = "N/A" }) {
  console.log(`${name}: ${marks} (${grade})`);
}
show({ name:"Aarav", marks:88 });
```

9.3 Modules (import / export)

```
// ■■ math.js ■■ (module file – export functions)
export function add(a, b) { return a + b; }
export function subtract(a, b) { return a - b; }
export const PI = 3.14159;

// Default export – one per file
export default function multiply(a, b) { return a * b; }

// ■■ app.js ■■ (import and use)
import multiply from "./math.js"; // default import
import { add, subtract, PI } from "./math.js"; // named imports
import * as MathUtils from "./math.js"; // import everything

console.log(add(5, 3)); // 8
console.log(multiply(4, 3)); // 12
console.log(MathUtils.PI); // 3.14159

// ■■ In HTML – use type="module" ■■
// <script type="module" src="app.js"></script>

// Re-export
export { add as addition } from "./math.js";
```

9.4 Promises

A **Promise** represents a value that may be available now, in the future, or never. It solves the 'callback hell' problem. States: **Pending** → **Fulfilled or Rejected**.

```
// Creating a Promise
const fetchData = new Promise((resolve, reject) => {
  let success = true;
  setTimeout(() => {
    if (success) {
      resolve({ name: "Aarav", marks: 88 });
    } else {
      reject(new Error("Failed to fetch data"));
    }
  }, 2000);
});

// Consuming a Promise
fetchData
  .then(data => {
    console.log("Data:", data.name);
    return data.marks; // pass to next .then()
  })
  .then(marks => {
    console.log("Marks:", marks);
  })
  .catch(error => {
    console.error("Error:", error.message);
  })
  .finally(() => {
    console.log("Done – runs always");
  });

// Promise methods
Promise.all([p1, p2, p3]) // waits for ALL to resolve
  .then(results => console.log(results));

Promise.allSettled([p1, p2, p3]) // waits for ALL (even rejected)
  .then(results => results.forEach(r => console.log(r.status)));

Promise.race([p1, p2, p3]) // resolves/rejects with FIRST one
  .then(first => console.log(first));

Promise.resolve("value") // creates immediately resolved Promise
Promise.reject(new Error("oops")) // creates immediately rejected Promise
```

9.5 async / await

async/await is syntactic sugar over Promises — it makes asynchronous code look and behave like synchronous code. Much easier to read than `.then()` chains.

```
// async function – always returns a Promise
async function getStudent() {
  return { name: "Aarav", marks: 88 };
}

// await – pause until Promise resolves (only inside async function)
async function fetchStudent(id) {
  try {
    // Simulated API call – replace with real fetch()
    let response = await fetch(`/api/students/${id}`);

    if (!response.ok) {
      throw new Error(`HTTP Error: ${response.status}`);
    }

    let data = await response.json();
    console.log("Student:", data.name);
    return data;
  } catch (error) {
    console.error("Failed:", error.message);
  } finally {
    console.log("Request complete");
  }
}

// Call async function
fetchStudent(1);

// Multiple awaits – sequential
async function loadAll() {
  let user = await fetchUser(1);
  let posts = await fetchPosts(user.id); // waits for user first
  return { user, posts };
}

// Parallel awaits (faster)
async function loadParallel() {
  let [user, posts] = await Promise.all([fetchUser(1), fetchPosts(1)]);
  return { user, posts };
}
```

9.6 Fetch API — Making HTTP Requests

```
// GET request – fetch public data
async function getUsers() {
  try {
    let response = await fetch("https://jsonplaceholder.typicode.com/users");
    let users    = await response.json();
    users.forEach(u => console.log(u.name, u.email));
  } catch (err) {
    console.error("Network error:", err);
  }
}
getUsers();

// POST request – send data to server
async function createPost(title, body) {
  let response = await fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body:    JSON.stringify({ title, body, userId: 1 }),
  });
  let newPost = await response.json();
  console.log("Created:", newPost.id);
}

// Display API data in HTML
async function displayUsers() {
  let res    = await fetch("https://jsonplaceholder.typicode.com/users");
  let users  = await res.json();
  let ul     = document.getElementById("user-list");
  ul.innerHTML = users.map(u =>
    `<li><strong>${u.name}</strong> – ${u.email}</li>`
  ).join("");
}
displayUsers();
```

Module 10 — Object-Oriented Programming (OOP)

10.1 Constructor Functions (Old Way)

```
// Constructor function – creates objects with same structure
function Student(name, age, marks) {
  this.name = name;
  this.age = age;
  this.marks = marks;
}

// Add methods via prototype (shared – not duplicated per instance)
Student.prototype.greet = function() {
  return `Hi, I am ${this.name}`;
};
Student.prototype.isPassed = function() {
  return this.marks >= 40;
};

// Create instances with 'new' keyword
let s1 = new Student("Aarav", 20, 88);
let s2 = new Student("Priya", 21, 92);
let s3 = new Student("Vikram", 20, 35);

console.log(s1.greet());      // Hi, I am Aarav
console.log(s3.isPassed());   // false

// instanceof check
console.log(s1 instanceof Student); // true
```

10.2 ES6 Classes (Modern OOP)

Classes are the modern, cleaner syntax for creating objects. Under the hood they still use prototypes.

```
class Student {
  // Constructor – runs when 'new Student()' is called
  constructor(name, age, marks) {
    this.name = name;
    this.age = age;
    this.marks = marks;
  }

  // Methods – automatically on prototype (not copied to each instance)
  greet() {
    return `Hello, I am ${this.name} from ForkInfosystems!`;
  }

  isPassed() {
    return this.marks >= 40;
  }

  getGrade() {
    if (this.marks >= 90) return "A+";
    if (this.marks >= 75) return "A";
    if (this.marks >= 60) return "B";
    if (this.marks >= 40) return "C";
    return "F";
  }

  // Getter – access like a property
  get info() {
    return `${this.name} (${this.age}) – Grade: ${this.getGrade()}`;
  }

  // Setter – validate before setting
  set newMarks(value) {
    if (value < 0 || value > 100)
      throw new Error("Marks must be 0-100");
    this.marks = value;
  }

  // Static method – called on class, not instance
  static passing() {
    return "40 marks required to pass";
  }
}

// Usage
let s1 = new Student("Aarav", 20, 88);
console.log(s1.greet());           // Hello, I am Aarav...
console.log(s1.info);              // Aarav (20) – Grade: A
s1.newMarks = 95;                  // uses setter
console.log(Student.passing());    // static – called on class
console.log(s1.isPassed());        // true
```

10.3 Inheritance — extends & super

```
// Base class (Parent)
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  introduce() {
    return `I am ${this.name}, age ${this.age}`;
  }
  toString() {
    return `Person: ${this.name}`;
  }
}

// Child class – inherits from Person
class Student extends Person {
  constructor(name, age, marks, course) {
    super(name, age); // MUST call super() first – runs Person constructor
    this.marks = marks;
    this.course = course;
  }

  // Override parent method
  introduce() {
    return `${super.introduce()}, studying ${this.course}`;
  }

  study() {
    return `${this.name} is studying ${this.course}`;
  }
}

class Teacher extends Person {
  constructor(name, age, subject, phone) {
    super(name, age);
    this.subject = subject;
    this.phone = phone;
  }
  teach() {
    return `${this.name} teaches ${this.subject}`;
  }
}

// Usage
let student = new Student("Aarav", 20, 88, "Web Development");
let teacher = new Teacher("Ranjit", 35, "JavaScript", "9172135355");

console.log(student.introduce()); // I am Aarav, age 20, studying Web Development
console.log(teacher.teach()); // Ranjit teaches JavaScript

// instanceof checks inheritance chain
console.log(student instanceof Student); // true
console.log(student instanceof Person); // true (inherited)
```

10.4 Encapsulation — Private Fields (ES2022)

```
class BankAccount {
  #balance;      // Private field – cannot access outside class
  #owner;

  constructor(owner, initialBalance) {
    this.#owner = owner;
    this.#balance = initialBalance;
  }

  deposit(amount) {
    if (amount <= 0) throw new Error("Amount must be positive");
    this.#balance += amount;
    return this.#balance;
  }

  withdraw(amount) {
    if (amount > this.#balance) throw new Error("Insufficient funds");
    this.#balance -= amount;
    return this.#balance;
  }

  get balance() { return this.#balance; } // controlled access
  get owner()   { return this.#owner; }

  toString() {
    return `${this.#owner}'s account: ■${this.#balance}`;
  }
}

let acc = new BankAccount("Aarav", 10000);
console.log(acc.deposit(5000)); // 15000
console.log(acc.withdraw(2000)); // 13000
console.log(acc.balance); // 13000
// console.log(acc.#balance); // SyntaxError – private!
console.log(acc.toString());
```

Module 11 — Error Handling, JSON & Local Storage

11.1 Types of Errors

Error Type	When it occurs	Example
SyntaxError	Code cannot be parsed	let x = ; (missing value)
ReferenceError	Using undefined variable	console.log(xyz) (xyz not declared)
TypeError	Wrong type operation	null.toUpperCase()
RangeError	Value out of range	new Array(-1)
URIError	Bad URI encoding	decodeURIComponent('%')

11.2 try / catch / finally

```
// Basic try-catch
try {
  let result = 10 / 0;          // Infinity – not an error in JS
  let bad    = null.toUpperCase(); // TypeError!
  console.log("This won't run");
} catch (error) {
  console.log("Error type:", error.name);      // TypeError
  console.log("Message:",   error.message);    // Cannot read...
  console.log("Stack:",     error.stack);       // Call stack trace
} finally {
  console.log("This ALWAYS runs – cleanup code here");
}

// Catching specific errors
function divide(a, b) {
  if (typeof a !== "number" || typeof b !== "number") {
    throw new TypeError("Arguments must be numbers");
  }
  if (b === 0) {
    throw new RangeError("Cannot divide by zero");
  }
  return a / b;
}

try {
  console.log(divide(10, 2)); // 5
  console.log(divide(10, 0)); // throws RangeError
} catch (e) {
  if (e instanceof RangeError) {
    console.log("Math error:", e.message);
  } else if (e instanceof TypeError) {
    console.log("Type error:", e.message);
  } else {
    throw e; // re-throw unexpected errors
  }
}

// Async error handling
async function fetchData(url) {
  try {
    let res = await fetch(url);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();
  } catch (err) {
    console.error("Fetch failed:", err.message);
    return null;
  }
}
```

11.3 Custom Error Classes

```
// Create custom error types
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = "ValidationError";
    this.field = field;
  }
}

class NetworkError extends Error {
  constructor(status, message) {
    super(message);
    this.name = "NetworkError";
    this.status = status;
  }
}

// Use custom errors
function validateAge(age) {
  if (typeof age !== "number") {
    throw new ValidationError("age", "Age must be a number");
  }
  if (age < 0 || age > 120) {
    throw new ValidationError("age", "Age must be between 0 and 120");
  }
  return true;
}

try {
  validateAge("twenty");
} catch (e) {
  if (e instanceof ValidationError) {
    console.log(`Field '${e.field}': ${e.message}`);
  }
}
```

11.4 JSON — JavaScript Object Notation

```
// JSON is a text format for storing/transmitting data
// Used for APIs, localStorage, config files

// JavaScript Object → JSON string
let student = { name:"Aarav", age:20, marks:88, hobbies:["Coding","Cricket"] };
let jsonStr = JSON.stringify(student);
// '{"name":"Aarav","age":20,"marks":88,"hobbies":["Coding","Cricket"]}'

// Pretty-print with indentation
let pretty = JSON.stringify(student, null, 2);
console.log(pretty);

// JSON string → JavaScript Object
let parsed = JSON.parse(jsonStr);
console.log(parsed.name);    // "Aarav"
console.log(parsed.hobbies[0]); // "Coding"

// Common use case – deep copy an object
let copy = JSON.parse(JSON.stringify(student));

// JSON.parse with error handling
try {
  let data = JSON.parse('{"name": "bad json"}'); // malformed
} catch (e) {
  console.error("Invalid JSON:", e.message);
}

// JSON rules:
// - Keys must be in double quotes
// - No functions, undefined, or Symbol values
// - null, string, number, boolean, array, object – allowed
```

11.5 localStorage & sessionStorage

```
// localStorage – persists even after browser is closed
// sessionStorage – cleared when tab/browser is closed

// SAVE data (only strings – use JSON.stringify for objects)
localStorage.setItem("username", "Aarav");
localStorage.setItem("marks", "88");
localStorage.setItem("student", JSON.stringify({ name:"Aarav", age:20 }));

// READ data
let username = localStorage.getItem("username"); // "Aarav"
let student = JSON.parse(localStorage.getItem("student"));
console.log(student.name); // "Aarav"

// DELETE
localStorage.removeItem("marks");
localStorage.clear(); // remove everything

// CHECK if key exists
if (localStorage.getItem("username")) {
  console.log("User is logged in");
}

// sessionStorage – same API, different lifetime
sessionStorage.setItem("cart", JSON.stringify([{id:1, qty:2}]));
let cart = JSON.parse(sessionStorage.getItem("cart"));

// Practical: save theme preference
function setTheme(theme) {
  document.body.className = theme;
  localStorage.setItem("theme", theme);
}

function loadTheme() {
  let saved = localStorage.getItem("theme") || "light";
  document.body.className = saved;
}

loadTheme(); // call on page load
```

Module 12 — Mini & Major Projects

12.1 Project 1 — To-Do List App

Covers: DOM manipulation, events, localStorage, array methods. Save as **todo.html**.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,initial-scale=1.0">
  <title>To-Do App – ForkInfosystems</title>
  <style>
    * { margin:0; padding:0; box-sizing:border-box; }
    body { font-family:'Segoe UI',Arial,sans-serif; background:#f0f4ff; min-height:100vh;
padding:30px 20px; }
    .app {
      max-width:550px; margin:0 auto;
      background:white; border-radius:16px;
      box-shadow:0 8px 30px rgba(0,0,0,0.12); overflow:hidden;
    }
    .app-header {
      background:linear-gradient(135deg,#1a237e,#e65100);
      color:white; padding:25px 30px; text-align:center;
    }
    .app-header h1 { font-size:24px; }
    .app-header p { opacity:0.85; font-size:13px; margin-top:4px; }
    .input-row {
      display:flex; gap:10px; padding:20px 25px;
      border-bottom:1px solid #eee;
    }
    #task-input {
      flex:1; padding:11px 15px; border:1.5px solid #c5cae9;
      border-radius:8px; font-size:15px; outline:none;
      transition:border 0.3s;
    }
    #task-input:focus { border-color:#1a237e; }
    #add-btn {
      padding:11px 20px; background:#1a237e; color:white;
      border:none; border-radius:8px; font-size:15px;
      cursor:pointer; transition:background 0.3s; white-space:nowrap;
    }
    #add-btn:hover { background:#e65100; }
    .filters {
      display:flex; gap:0; padding:0 25px 15px; margin-top:10px;
    }
    .filter-btn {
      flex:1; padding:8px; border:1px solid #ddd; background:white;
      cursor:pointer; font-size:13px; transition:all 0.2s;
    }
    .filter-btn:first-child { border-radius:6px 0 0 6px; }
    .filter-btn:last-child { border-radius:0 6px 6px 0; }
    .filter-btn.active { background:#1a237e; color:white; border-color:#1a237e; }
    #todo-list { list-style:none; padding:0 25px 20px; }
    .todo-item {
      display:flex; align-items:center; gap:12px;
      padding:13px 15px; margin-bottom:10px;
      background:#f9f9f9; border-radius:10px;
      border-left:4px solid #1a237e;
    }
```

```

    transition:all 0.3s; animation:slideIn 0.3s ease;
  }
  @keyframes slideIn {
    from { opacity:0; transform:translateY(-10px); }
    to   { opacity:1; transform:translateY(0); }
  }
  .todo-item.completed { border-left-color:#4caf50; opacity:0.7; }
  .todo-item.completed .task-text { text-decoration:line-through; color:#aaa; }
  .check-btn {
    width:24px; height:24px; border-radius:50%;
    border:2px solid #1a237e; background:white;
    cursor:pointer; display:flex; align-items:center;
    justify-content:center; font-size:14px; flex-shrink:0;
    transition:all 0.2s;
  }
  .todo-item.completed .check-btn { background:#4caf50; border-color:#4caf50; color:white; }
  .task-text { flex:1; font-size:15px; color:#333; }
  .delete-btn {
    background:none; border:none; cursor:pointer;
    color:#e53935; font-size:18px; padding:4px;
    border-radius:4px; transition:background 0.2s;
  }
  .delete-btn:hover { background:#fce4ec; }
  .stats {
    padding:12px 25px; background:#f5f7ff;
    border-top:1px solid #eee; font-size:13px; color:#666;
    display:flex; justify-content:space-between; align-items:center;
  }
  .clear-btn {
    background:none; border:none; color:#e65100;
    cursor:pointer; font-size:13px;
  }
  .empty { text-align:center; padding:30px; color:#bbb; font-size:15px; }
</style>
</head>
<body>
<div class="app">
  <div class="app-header">
    <h1>&#9989; To-Do App</h1>
    <p>ForkInfosystems JavaScript Project</p>
  </div>

  <div class="input-row">
    <input type="text" id="task-input" placeholder="Add a new task..." maxlength="100">
    <button id="add-btn">+ Add</button>
  </div>

  <div class="filters">
    <button class="filter-btn active" data-filter="all">All</button>
    <button class="filter-btn" data-filter="active">Active</button>
    <button class="filter-btn" data-filter="completed">Completed</button>
  </div>
</div>

```

```
</div>

<ul id="todo-list"></ul>

<div class="stats">
  <span id="stats-text">0 tasks</span>
  <button class="clear-btn" id="clear-completed">Clear Completed</button>
</div>
</div>

<script>
  // State
  let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
  let filter = "all";

  // Save to localStorage
  function save() {
    localStorage.setItem("tasks", JSON.stringify(tasks));
  }

  // Render tasks
  function render() {
    const list = document.getElementById("todo-list");
    const filtered = tasks.filter(t => {
      if (filter === "active") return !t.done;
      if (filter === "completed") return t.done;
      return true;
    });

    if (filtered.length === 0) {
      list.innerHTML = `<li class="empty">No tasks to show &#128591;</li>`;
    } else {
      list.innerHTML = filtered.map(t => `
        <li class="todo-item ${t.done ? "completed" : ""}" data-id="${t.id}">
          <button class="check-btn">${t.done ? "&#10003;" : ""}</button>
          <span class="task-text">${t.text}</span>
          <button class="delete-btn">&#10005;</button>
        </li>
      `).join("");
    }
  }

  // Stats
  const done = tasks.filter(t => t.done).length;
  document.getElementById("stats-text").textContent =
    `${tasks.length} tasks — ${done} completed`;
}

// Add task
function addTask() {
  const input = document.getElementById("task-input");
  const text = input.value.trim();
  if (!text) return;
```

```
    tasks.push({ id: Date.now(), text, done: false });
    input.value = "";
    save();
    render();
  }

  // Event: Add button & Enter key
  document.getElementById("add-btn").addEventListener("click", addTask);
  document.getElementById("task-input").addEventListener("keydown", e => {
    if (e.key === "Enter") addTask();
  });

  // Event: Click on list (delegation)
  document.getElementById("todo-list").addEventListener("click", e => {
    const item = e.target.closest(".todo-item");
    if (!item) return;
    const id = Number(item.dataset.id);

    if (e.target.classList.contains("check-btn") ||
        e.target.classList.contains("task-text")) {
      tasks = tasks.map(t => t.id === id ? { ...t, done: !t.done } : t);
    }
    if (e.target.classList.contains("delete-btn")) {
      tasks = tasks.filter(t => t.id !== id);
    }
    save(); render();
  });

  // Filter buttons
  document.querySelectorAll(".filter-btn").forEach(btn => {
    btn.addEventListener("click", () => {
      document.querySelectorAll(".filter-btn").forEach(b => b.classList.remove("active"));
      btn.classList.add("active");
      filter = btn.dataset.filter;
      render();
    });
  });

  // Clear completed
  document.getElementById("clear-completed").addEventListener("click", () => {
    tasks = tasks.filter(t => !t.done);
    save(); render();
  });

  // Initial render
  render();
</script>
</body>
</html>
```

12.2 Project 2 — Quiz App

Covers: Arrays of objects, DOM, events, scoring, timer. Save as **quiz.html**.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,initial-scale=1.0">
  <title>JS Quiz — ForkInfosystems</title>
  <style>
    * { margin:0; padding:0; box-sizing:border-box; }
    body {
      font-family:'Segoe UI',Arial,sans-serif;
      background:linear-gradient(135deg,#1a237e,#e65100);
      min-height:100vh; display:flex;
      align-items:center; justify-content:center; padding:20px;
    }
    .quiz-card {
      background:white; border-radius:16px; padding:35px;
      max-width:560px; width:100%;
      box-shadow:0 20px 60px rgba(0,0,0,0.3);
    }
    .header { display:flex; justify-content:space-between; margin-bottom:20px; }
    .progress { font-size:13px; color:#888; }
    .timer { font-size:13px; color:#e65100; font-weight:bold; }
    .progress-bar {
      height:6px; background:#eee; border-radius:3px; margin-bottom:25px;
    }
    .progress-fill {
      height:100%; background:linear-gradient(to right,#1a237e,#e65100);
      border-radius:3px; transition:width 0.4s ease;
    }
    .question { font-size:20px; font-weight:bold; color:#1a237e; margin-bottom:25px; line-height:1.4; }
    .options { display:flex; flex-direction:column; gap:12px; }
    .option-btn {
      padding:14px 18px; border:2px solid #e0e0e0;
      border-radius:10px; background:white; font-size:15px;
      cursor:pointer; text-align:left; transition:all 0.25s;
    }
    .option-btn:hover:not(:disabled) { border-color:#1a237e; background:#eef0fb; }
    .option-btn.correct { border-color:#4caf50; background:#e8f5e9; color:#2e7d32; font-weight:bold; }
    .option-btn.incorrect { border-color:#e53935; background:#fce4ec; color:#b71c1c; }
    .next-btn {
      display:none; width:100%; padding:14px; margin-top:20px;
      background:#1a237e; color:white; border:none; border-radius:10px;
      font-size:16px; cursor:pointer; transition:background 0.3s;
    }
    .next-btn:hover { background:#e65100; }
    .result { text-align:center; }
    .score-circle {
      width:130px; height:130px; border-radius:50%;
      background:linear-gradient(135deg,#1a237e,#e65100);
      display:flex; align-items:center; justify-content:center;
      margin:0 auto 20px; color:white;
    }
```

```
    }
    .score-circle span { font-size:36px; font-weight:bold; }
    .result h2 { font-size:24px; color:#1a237e; margin-bottom:10px; }
    .result p { color:#666; margin-bottom:20px; }
    .restart-btn {
      padding:13px 35px; background:#e65100; color:white;
      border:none; border-radius:30px; font-size:15px;
      cursor:pointer; font-weight:bold;
    }
  </style>
</head>
<body>
<div class="quiz-card" id="quiz-card">
  <div class="header">
    <span class="progress" id="progress">Question 1 of 5</span>
    <span class="timer" id="timer">&#9201; 15s</span>
  </div>
  <div class="progress-bar">
    <div class="progress-fill" id="progress-fill" style="width:0%"></div>
  </div>
  <div class="question" id="question"></div>
  <div class="options" id="options"></div>
  <button class="next-btn" id="next-btn">Next &rarr;</button>
</div>

<script>
const questions = [
  {
    q: "Which keyword is used to declare a variable that cannot be reassigned?",
    options: ["var", "let", "const", "static"],
    answer: 2
  },
  {
    q: "What does the === operator check?",
    options: ["Value only", "Type only", "Value AND type", "Neither"],
    answer: 2
  },
  {
    q: "Which array method creates a NEW array without changing the original?",
    options: ["push()", "map()", "sort()", "splice()"],
    answer: 1
  },
  {
    q: "What is a closure in JavaScript?",
    options: [
      "A way to close the browser",
      "A function that remembers its outer scope",
      "An error handling technique",
      "A CSS-like selector"
    ],
    answer: 1
  },
]
```

```
{
  q: "Which method converts a JavaScript object to a JSON string?",
  options: ["JSON.parse()", "JSON.stringify()", "JSON.convert()", "JSON.toString()"],
  answer: 1
},
];

let current = 0, score = 0, timer, timeLeft;

function startTimer() {
  timeLeft = 15;
  document.getElementById("timer").textContent = `■ ${timeLeft}s`;
  timer = setInterval(() => {
    timeLeft--;
    document.getElementById("timer").textContent = `■ ${timeLeft}s`;
    if (timeLeft <= 0) { clearInterval(timer); autoNext(); }
  }, 1000);
}

function loadQuestion() {
  clearInterval(timer);
  const q = questions[current];
  const pct = (current / questions.length) * 100;
  document.getElementById("progress").textContent = `Question ${current+1} of ${questions.
length}`;
  document.getElementById("progress-fill").style.width = pct + "%";
  document.getElementById("question").textContent = q.q;
  document.getElementById("next-btn").style.display = "none";

  const opts = document.getElementById("options");
  opts.innerHTML = q.options.map((opt, i) =>
    `<button class="option-btn" data-i="${i}">${opt}</button>`
  ).join("");

  opts.querySelectorAll(".option-btn").forEach(btn => {
    btn.addEventListener("click", () => selectAnswer(Number(btn.dataset.i)));
  });
  startTimer();
}

function selectAnswer(chosen) {
  clearInterval(timer);
  const q = questions[current];
  const btns = document.querySelectorAll(".option-btn");
  btns.forEach(b => b.disabled = true);
  btns[q.answer].classList.add("correct");
  if (chosen !== q.answer) btns[chosen].classList.add("incorrect");
  else score++;
  document.getElementById("next-btn").style.display = "block";
}

function autoNext() {
```

```
const q = questions[current];
const btns = document.querySelectorAll(".option-btn");
btns.forEach(b => b.disabled = true);
btns[q.answer].classList.add("correct");
document.getElementById("next-btn").style.display = "block";
}

document.getElementById("next-btn").addEventListener("click", () => {
  current++;
  if (current < questions.length) { loadQuestion(); }
  else { showResult(); }
});

function showResult() {
  const pct = Math.round((score / questions.length) * 100);
  const msg = pct >= 80 ? "Excellent! 🎉" : pct >= 60 ? "Good job! 👍" : "Keep practising! 📚";
  document.getElementById("quiz-card").innerHTML = `
    <div class="result">
      <div class="score-circle"><span>${pct}%</span></div>
      <h2>${msg}</h2>
      <p>You scored <strong>${score}</strong> out of ${questions.length}</strong></p>
      <button class="restart-btn" onclick="location.reload()">🔄 Restart Quiz</button>
    </div>
  `;
}

loadQuestion();
</script>
</body>
</html>
```

12.3 Project 3 — Weather App (Fetch API)

Covers: fetch API, async/await, JSON, DOM, error handling. Uses OpenWeatherMap free API. Save as **weather.html**.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,initial-scale=1.0">
  <title>Weather App</title>
  <style>
    * { margin:0; padding:0; box-sizing:border-box; }
    body {
      font-family:'Segoe UI',Arial,sans-serif; min-height:100vh;
      background:linear-gradient(135deg,#0f2027,#203a43,#2c5364);
      display:flex; align-items:center; justify-content:center; padding:20px;
    }
    .card {
      background:rgba(255,255,255,0.1); backdrop-filter:blur(10px);
      border:1px solid rgba(255,255,255,0.2);
      border-radius:20px; padding:35px; max-width:420px;
      width:100%; color:white; text-align:center;
    }
    h1 { font-size:22px; margin-bottom:25px; opacity:0.9; }
    .search-row {
      display:flex; gap:10px; margin-bottom:25px;
    }
    #city-input {
      flex:1; padding:12px 16px; border:none; border-radius:10px;
      background:rgba(255,255,255,0.15); color:white;
      font-size:15px; outline:none;
    }
    #city-input::placeholder { color:rgba(255,255,255,0.6); }
    #search-btn {
      padding:12px 18px; background:#e65100; border:none;
      border-radius:10px; color:white; font-size:15px; cursor:pointer;
    }
    .weather-icon { font-size:80px; margin:10px 0; }
    .temperature { font-size:60px; font-weight:700; margin:5px 0; }
    .city-name { font-size:22px; opacity:0.9; margin-bottom:5px; }
    .description { font-size:15px; opacity:0.75; text-transform:capitalize; margin-bottom:20px; }
    .details {
      display:grid; grid-template-columns:1fr 1fr 1fr;
      gap:12px; margin-top:20px;
    }
    .detail-box {
      background:rgba(255,255,255,0.1); border-radius:12px; padding:12px;
    }
    .detail-box .label { font-size:11px; opacity:0.7; margin-bottom:4px; }
    .detail-box .value { font-size:18px; font-weight:bold; }
    .error { color:#ff8a80; font-size:15px; margin-top:10px; }
    .loading { opacity:0.7; font-size:15px; margin-top:10px; }
    .hidden { display:none; }
  </style>
</head>
<body>
```

```
<div class="card">
  <h1>Weather App</h1>
  <div class="search-row">
    <input type="text" id="city-input" placeholder="Enter city name..." value="Karad">
    <button id="search-btn">Search</button>
  </div>
  <p class="loading hidden" id="loading">Fetching weather...</p>
  <p class="error hidden" id="error-msg"></p>
  <div id="weather-result" class="hidden">
    <div class="weather-icon" id="w-icon"></div>
    <div class="temperature" id="w-temp"></div>
    <div class="city-name" id="w-city"></div>
    <div class="description" id="w-desc"></div>
    <div class="details">
      <div class="detail-box">
        <div class="label">Humidity</div>
        <div class="value" id="w-humidity"></div>
      </div>
      <div class="detail-box">
        <div class="label">Wind</div>
        <div class="value" id="w-wind"></div>
      </div>
      <div class="detail-box">
        <div class="label">Feels Like</div>
        <div class="value" id="w-feels"></div>
      </div>
    </div>
  </div>
</div>

<script>
  // ■■ Get free API key from: https://openweathermap.org/api
  // Replace "YOUR_API_KEY" with your actual key
  const API_KEY = "YOUR_API_KEY";

  const weatherIcons = {
    Clear:      "&#9728;",
    Clouds:     "&#9925;",
    Rain:       "&#127783;",
    Drizzle:    "&#127784;",
    Thunderstorm: "&#9928;",
    Snow:       "&#10052;",
    Mist:       "&#127786;",
    Fog:        "&#127786;",
    Haze:       "&#127783;",
  };

  async function getWeather(city) {
    const loading = document.getElementById("loading");
    const result = document.getElementById("weather-result");
    const errMsg = document.getElementById("error-msg");
```

```
loading.classList.remove("hidden");
result.classList.add("hidden");
errMsg.classList.add("hidden");

try {
  const url = `https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${API_KEY}&units=metric`;
  const res = await fetch(url);
  const data = await res.json();

  if (data.cod !== 200) throw new Error(data.message);

  // Update UI
  const icon = weatherIcons[data.weather[0].main] || "&#127760;";
  document.getElementById("w-icon").innerHTML = icon;
  document.getElementById("w-temp").textContent = `${Math.round(data.main.temp)}°C`;
  document.getElementById("w-city").textContent = `${data.name}, ${data.sys.country}`;

  document.getElementById("w-desc").textContent = data.weather[0].description;
  document.getElementById("w-humidity").textContent = `${data.main.humidity}%`;
  document.getElementById("w-wind").textContent = `${data.wind.speed} m/s`;
  document.getElementById("w-feels").textContent = `${Math.round(data.main.feels_like)}°C`;

  result.classList.remove("hidden");
} catch (err) {
  errMsg.textContent = `Error: ${err.message}`;
  errMsg.classList.remove("hidden");
} finally {
  loading.classList.add("hidden");
}

document.getElementById("search-btn").addEventListener("click", () => {
  const city = document.getElementById("city-input").value.trim();
  if (city) getWeather(city);
});

document.getElementById("city-input").addEventListener("keydown", (e) => {
  if (e.key === "Enter") {
    const city = e.target.value.trim();
    if (city) getWeather(city);
  }
});

// Load default city on start
getWeather("Karad");
</script>
</body>
</html>
```